Google Cloud Associate Engineer Course Content 2026

☁ 1. What is Cloud Computing?

Cloud Computing means using IT resources (servers, storage, databases, networking, software) over the internet instead of buying and managing physical hardware.

Instead of:

- Buying servers
- Maintaining data centers
- Managing hardware

You simply **rent resources from cloud providers** and pay only for what you use.

✅ **Benefits:**

- Cost saving (Pay as you go)
- High scalability
- High availability
- Global access

Example: Hosting a website without buying a physical server.

🌍 2. Google Cloud Computing Services (GCP Services)

Google Cloud Platform (GCP) provides many cloud services. Main categories:

🔹 Compute Services (Run Applications)

- Compute Engine – Virtual machines (like AWS EC2)
- App Engine – Platform to deploy apps easily
- Cloud Functions – Serverless computing
- Kubernetes Engine (GKE) – Manage containers

💾 3. GCP Storage and Database Services

🔹 **Storage Services:**

- **Cloud Storage** – Object storage (like AWS S3)
- **Persistent Disk** – Block storage for VMs
- **Filestore** – File storage

🔹 **Database Services:**

- **Cloud SQL** – Managed MySQL, PostgreSQL, SQL Server
- **Cloud Spanner** – Globally distributed relational DB
- **Firestore** – NoSQL document database
- **Bigtable** – Large-scale NoSQL database

🌐 4. Networking with Google Cloud

Networking helps connect resources securely.

Important Services:

- **VPC (Virtual Private Cloud)** – Create private network
- **Cloud Load Balancing** – Distribute traffic
- **Cloud CDN** – Fast content delivery
- **Cloud DNS** – Manage domain names
- **VPN & Interconnect** – Connect on-premises to GCP

GCP networking is global and highly scalable.

🚀 5. Additional Services in Google Cloud

Some other important services:

- **IAM** – Manage users and permissions
- **Cloud Monitoring & Logging** – Track performance
- **Cloud AI & ML Services** – Machine learning tools
- **BigQuery** – Data analytics
- **Pub/Sub** – Messaging service

🚀 Getting Started with Google Cloud Console & Cloud Shell

1□.Introduction to the Google Cloud Console

The **Google Cloud Console** is a web-based interface used to manage all GCP services.

It allows you to:

- Create and manage Virtual Machines
- Configure networking
- Create storage buckets
- Manage databases
- Monitor resources

- Set IAM permissions

◆ **Key Parts of Console:**

- **Navigation Menu (☰)** – Access all services
- **Dashboard** – Overview of your project
- **Project Selector** – Choose your active project
- **Search Bar** – Quickly find services
- **Cloud Shell Icon (>_)** – Open terminal

☞ In simple words:
Cloud Console = Control panel of Google Cloud.

2. Starting Cloud Shell

Cloud Shell is a browser-based terminal provided by GCP.

It gives you:

- Linux environment
- Pre-installed Cloud SDK
- 5GB persistent storage
- Secure access to your project

◆ **How to Start:**

1. Click **Cloud Shell icon (>_)**
2. Wait for terminal to initialize
3. Start typing commands

☞ No need to install anything on your computer.

Cloud Shell = Online Linux terminal inside GCP.

3.Launch a Code Editor

Inside Cloud Shell, you can open a built-in **Code Editor**.

How to Open:

- Click **Open Editor** button
  OR
- Use command:

```
cloudshell edit .
```

Features:

- Create & edit files
- Write code (Python, JS, etc.)
- Edit configuration files
- Deploy applications

☞ It works like a simple VS Code inside browser.

4.Introduction to Cloud SDK Commands

Cloud SDK is a set of command-line tools to manage GCP.

Main command:

```
>>   gcloud
```

It helps you:

- Create VM instances
- Manage projects
- Deploy apps
- Configure networks
- Manage IAM

Common Commands:

Check active project:

```
gcloud config list
```

List compute instances:

```
gcloud compute instances list
```

Set project:

```
gcloud config set project PROJECT_ID
```

☞ gcloud = Command-line tool to control Google Cloud.

5.Exercise: Enter Linux Commands and Gcloud Commands

◆ Basic Linux Commands (in Cloud Shell)

Check current directory:

```
pwd
```

List files:

```
ls
```

Create folder:

```
mkdir test-folder
```

Change directory:

```
cd test-folder
```

Create file:

```
touch file.txt
```

◆ Basic Gcloud Commands Practice

Check current project:

```
gcloud config get-value project
```

List zones:

```
gcloud compute zones list
```

List VM instances:

```
gcloud compute instances list
```

🏦 Enabling API & Resource Hierarchy in Google Cloud

1□.Create a Billing Account

A **Billing Account** is required to use most Google Cloud services.

Even if you use free tier or free credits, billing setup is mandatory.

◆ **Why Billing Account is Important?**

- To activate paid services
- To track usage and cost
- To manage invoices
- To link projects with payment method

◆ **Steps to Create:**

1. Go to **Google Cloud Console**
2. Open **Billing Section**
3. Click **Create Billing Account**
4. Enter payment details
5. Link billing account to your project

☞ Without billing, most services cannot run.

2.Enable GCP Service APIs

In Google Cloud, services work through **APIs**.
Before using a service (like Compute Engine), you must enable its API.

◆ **What is API?**

API (Application Programming Interface) allows your project to communicate with GCP services.

For example:

- Compute Engine API → To create VM
- Cloud Storage API → To create bucket

◆ **How to Enable API:**

1. Go to **APIs & Services**
2. Click **Enable APIs and Services**
3. Search for service (e.g., Compute Engine)
4. Click **Enable**

☞ If API is not enabled, service will not work.

3.Introduction to the Resource Hierarchy

Google Cloud organizes resources in a structure called **Resource Hierarchy**.

Structure:

```
Organization
    ↓
Folder
    ↓
Project
    ↓
Resources (VMs, Buckets, Databases)
```

Organization

- Top-level container
- Represents a company or domain

◆ **Folder**

- Used to group projects
- Helpful for departments (HR, IT, Dev)

◆ **Project**

- Main working unit in GCP
- Every resource must belong to a project

◆ **Resources**

- VM Instances
- Storage Buckets
- Databases
- Networks

4.Setting Up Billing (Linking to Project)

After creating a billing account, you must link it to your project.

Steps:

1. Go to **Billing**
2. Select your Billing Account
3. Click **Manage Billing Accounts**
4. Link your Project

Why This is Important:

- Allows services to run
- Tracks project cost separately
- Helps in budget management

🔐 Identity & Access Management (IAM) – Google Cloud

1□.Introduction to Identity & Access Management (IAM)

IAM (Identity & Access Management) is used to control **who can access what resources** in Google Cloud.

It helps you:

- Manage users
- Control permissions
- Secure resources
- Follow security best practices

☞ In simple words:
IAM decides **WHO can do WHAT on WHICH resource**.

◆ **IAM Works on 3 Things:**

- **Identity (Who)** → User, Group, Service Account
- **Role (What they can do)** → Set of permissions
- **Resource (Where)** → Project, Folder, or Resource

Example:
Give a developer permission to create VM but not delete them.

2.Viewing IAM Assignments

IAM assignments show:

- Which users are added
- What roles they have
- On which project or resource

◆ **How to View:**

1. Go to **IAM & Admin**
2. Click **IAM**
3. See list of members and assigned roles

You can check:

- Email ID (User)
- Role assigned
- Level (Project/Folder/Organization)

3.Overview of IAM Roles

A **Role** is a collection of permissions.

There are 3 main types of roles:

◆ **1. Basic Roles**

- Owner
- Editor
- Viewer

⚠ Not recommended for production (too broad access)

◆ **2. Predefined Roles**

- Created by Google
- More specific
- Example:
    - Compute Admin
    - Storage Admin
    - Network Admin

Recommended for most cases.

◆ **3. Custom Roles**

- Created by you
- Choose specific permissions
- Useful for special requirements

4□.Creating Custom Roles

Custom roles allow you to:

- Select only required permissions
- Follow least privilege principle

Steps:

1. Go to **IAM & Admin**
2. Click **Roles**
3. Click **Create Role**
4. Add permissions
5. Save

Example:
Create role that allows:

- Start VM
- Stop VM
  But not delete VM

5.Assigning IAM Roles

To give access to a user:

Steps:

1. Go to **IAM**
2. Click **Grant Access**
3. Enter user email
4. Select role
5. Click Save

You can assign roles at:

- Organization level
- Folder level
- Project level
- Resource level

6□.IAM Best Practices

Follow these security rules:

✅ 1. Principle of Least Privilege

Give only required permissions.

✅ 2. Avoid Basic Roles

Use predefined or custom roles.

✅ 3. Use Groups Instead of Individuals

Easier to manage access.

✅ 4. Regularly Review Access

Remove unused users.

✅ 5. Use Service Accounts Properly

For applications, not human users.

✅ 6. Enable Audit Logs

Monitor access and changes.

🗄 Starting a Virtual Machine (VM) in Google Cloud

A **Virtual Machine (VM)** is a virtual computer that runs on Google's cloud infrastructure.
You can use it like a normal computer (Linux/Windows) to run applications.

1.Create a Virtual Machine using Cloud Console

Cloud Console is the web interface of GCP.

◆ **Steps:**

1. Go to **Google Cloud Console**
2. Select your **Project**
3. Go to **Compute Engine → VM Instances**
4. Click **Create Instance**
5. Enter:
    - VM Name
    - Region & Zone
    - Machine Type (e.g., e2-micro)
    - Boot Disk (Linux/Windows)
6. Click **Create**

Your VM will start in a few seconds.

2.Create a Virtual Machine using Cloud Shell

Cloud Shell is a browser-based terminal.

◆ **Command Example:**

```
gcloud compute instances create my-vm \
--zone=us-central1-a \
--machine-type=e2-micro \
--image-family=debian-11 \
--image-project=debian-cloud
```

This command:

- Creates VM named **my-vm**
- Uses **e2-micro machine**
- Uses **Debian Linux image**

3.Installing the Cloud SDK Locally

Cloud SDK allows you to run **gcloud commands** from your own computer.

◆ **Steps:**

1. Download Cloud SDK from Google Cloud website
2. Install it on Windows/Linux/Mac
3. Open terminal and run:

```
gcloud init
```

4. Login with your Google account
5. Select project

Now you can manage GCP from your local machine.

4.Create a Virtual Machine using Cloud SDK (Local System)

After installing SDK:

Example Command:

```
gcloud compute instances create test-vm \
--machine-type=e2-micro \
--zone=us-central1-a
```

You can also add:

- Disk size
- Network settings
- Tags
- Firewall rules

5.Accessing a Virtual Machine

Once VM is created, you can access it.

◆ **Option 1: Browser SSH**

- Go to **VM Instances**

- Click **SSH**
- Terminal opens in browser

◆ **Option 2: Using gcloud**

```
gcloud compute ssh my-vm --zone=us-central1-a
```

◆ **Option 3: Using External SSH Tool**

- Use public IP
- Use SSH key
- Connect via PuTTY (Windows)

6.Exercise: Create a Virtual Machine

Practice Task:

1. Create VM using Cloud Console
2. Connect via SSH
3. Run Linux command:
4. `sudo apt update`
5. Install Apache:
6. `sudo apt install apache2`
7. Open browser and enter VM public IP
8. Verify web server is running

🗄 Advanced Virtual Machine Configuration – Google Cloud

1□.View Virtual Machine Inventory

VM Inventory means viewing all virtual machines in your project.

◆ **Using Cloud Console:**

- Go to **Compute Engine → VM Instances**
- You can see:
  - VM Name
  - Zone

- Machine Type
- Internal/External IP
- Status (Running/Stopped)

Using Command:

```
gcloud compute instances list
```

☞ Helps monitor and manage all VMs easily.

2.Machine Configurations and Machine Types

Machine type defines:

- CPU (vCPUs)
- Memory (RAM)

◆ **Common Machine Families:**

- **E2** → General purpose (low cost)
- **N2 / N2D** → Balanced performance
- **C2** → Compute optimized (high CPU)
- **M1 / M2** → Memory optimized

Example:

- e2-micro → 2 vCPU (shared), small RAM
- e2-standard-4 → 4 vCPU, 16GB RAM

☞ Choose based on workload (small app, database, heavy computing).

3.Attaching Additional Disks

By default, VM has a **boot disk** (OS disk).You can attach extra disks for storing data.

◆ **Types of Disks:**

- Standard Persistent Disk (HDD)
- SSD Persistent Disk

- Balanced Disk

◆ **Steps:**

1. Go to VM
2. Click **Edit**
3. Add **New Disk**
4. Save

☞ Useful for:

- Storing application data
- Increasing storage without deleting VM

4.Virtual Machines with GPUs

GPU VMs are used for:

- Machine Learning
- AI Training
- Video processing
- High-performance computing

◆ **Supported GPUs:**

- NVIDIA Tesla series

When creating VM:

- Select **GPU option**
- Choose number of GPUs

⚠ GPU VMs are more expensive.

5.Custom Machine Type

Instead of predefined machine types, you can create **Custom Machine Type**.

You can manually choose:

- Number of vCPUs
- Amount of RAM

Example:

- 3 vCPUs + 10GB RAM

☞ Useful when predefined types don’t match your exact needs.

6.Creating Snapshots

A **Snapshot** is a backup of a disk.

Used for:

- Backup
- Disaster recovery
- Creating new VM from existing disk

◆ **Steps:**

1. Go to **Disks**
2. Select disk
3. Click **Create Snapshot**

Snapshots are stored in Cloud Storage.

7.Using Spot Instances

Spot VM (previously Preemptible VM) is a low-cost VM.

◆ **Features:**

- Up to 60–90% cheaper
- Can be stopped anytime by Google
- Max runtime limit

◆ **Use Cases:**

- Batch jobs
- Testing
- Short-term workloads
- CI/CD pipelines

⚠ Not suitable for critical production workloads.

📄 Compute Engine Instance Group – Google Cloud

Instance Group means **a collection of Virtual Machines (VMs)** managed together.

It helps in:

- High availability
- Auto scaling
- Load balancing

1.Introduction to Managed Instance Groups (MIG)

A **Managed Instance Group (MIG)** automatically manages multiple identical VMs.

◆ **Key Features:**

- Automatically creates VMs from a template
- Replaces failed VMs
- Supports autoscaling
- Works with Load Balancer

◆ **Types of Instance Groups:**

1. **Managed Instance Group (MIG)** → Auto managed (recommended)
2. **Unmanaged Instance Group** → Manual control

☞ MIG is mostly used in production environments.

2.Creating a Managed Instance Group

Before creating MIG, you need an **Instance Template**.

◆ **Step 1: Create Instance Template**

1. Go to **Compute Engine → Instance Templates**
2. Click **Create Template**
3. Select:
    - Machine type
    - Boot disk
    - Firewall rules
4. Save template

◆ **Step 2: Create Managed Instance Group**

1. Go to **Instance Groups**
2. Click **Create Instance Group**
3. Select:
    - Instance template
    - Zone or Region
    - Number of instances
4. Click **Create**

Now Google will automatically create multiple VMs.

☞ All VMs will be identical.

3.Autoscaling with Managed Instance Group

Autoscaling means automatically increasing or decreasing VM count based on load.

◆ **How it Works:**

- If CPU usage increases → Add more VMs
- If CPU usage decreases → Remove extra VMs

◆ **Configure Autoscaling:**

1. Open Instance Group
2. Click **Edit**
3. Enable **Autoscaling**
4. Set:
    - Minimum instances
    - Maximum instances
    - CPU utilization target (e.g., 60%)

Example:

- Min: 2 VMs
- Max: 5 VMs
- CPU Target: 70%

☞ Helps in cost optimization and performance improvement.

4.Exercise: Creating an Instance Group

Practice Steps:

1□.Create an Instance Template
2□.Create a Managed Instance Group with 2 VMs
3□.Enable Autoscaling (Min: 2, Max: 4, CPU: 60%)
4□.Check VM instances created automatically
5□.Stop one VM → See if it auto recreates

☸ Introduction to Kubernetes Engine (GKE)

1□.Getting Started with Google Kubernetes Engine (GKE)

Google Kubernetes Engine (GKE) is a managed service that runs **Kubernetes clusters** on Google Cloud.

☞ Kubernetes helps you run and manage **containers** (like Docker containers).

Instead of managing servers manually, Kubernetes:

- Deploys applications
- Scales applications
- Manages failures automatically

In simple words:
GKE = Managed Kubernetes service by Google.

2.Kubernetes Architecture (Basic Understanding)

Kubernetes works using two main parts:

◆ **1. Control Plane (Master)**

- Manages cluster
- Schedules containers
- Monitors health

Main Components:

- API Server
- Scheduler
- Controller Manager
- etcd (stores cluster data)

2. Worker Nodes

- Run actual applications
- Contain Pods

Each Node has:

- Kubelet
- Container runtime (Docker/containerd)

Important Kubernetes Terms:

- **Cluster** → Group of nodes
- **Node** → Virtual machine
- **Pod** → Smallest unit (contains container)
- **Deployment** → Manages replicas of pods
- **Service** → Exposes application

3.Introduction to Anthos

Anthos is Google's multi-cloud and hybrid cloud platform.

It allows you to:

- Manage Kubernetes clusters across:
  - Google Cloud
  - AWS
  - On-premises

☞ Anthos helps manage multiple environments from one place.

In simple words:
Anthos = Multi-cloud Kubernetes management platform.

4.Creating an AutoPilot Mode Cluster

GKE has two modes:

- Autopilot
- Standard

◆ **Autopilot Mode**

In Autopilot:

- Google manages nodes
- Automatic scaling
- Pay per Pod
- Less configuration

Steps:

1. Go to **Kubernetes Engine**
2. Click **Create Cluster**
3. Select **Autopilot Mode**
4. Choose region
5. Click Create

☞ Best for beginners and simple workloads.

5.Creating a Standard Mode Cluster

In Standard Mode:

- You manage nodes
- You choose machine type
- More control
- More flexibility

Steps:

1. Go to **Kubernetes Engine**
2. Click **Create Cluster**

3. Select **Standard Mode**
4. Configure:
    - Node pool
    - Machine type
    - Number of nodes
5. Create

☞ Best for production & advanced users.

6□.Deploying a Cluster using Cloud Shell

You can create cluster using gcloud command:

Example (Autopilot):

```
gcloud container clusters create-auto my-cluster \
--region=us-central1
```

Example (Standard):

```
gcloud container clusters create my-cluster \
--zone=us-central1-a \
--num-nodes=3
```

To connect cluster:

```
gcloud container clusters get-credentials my-cluster
--zone=us-central1-a
```

7□.Deploying Applications in Kubernetes Engine

After cluster is ready:

Step 1: Deploy Application

Example:

```
kubectl create deployment my-app --image=nginx
```

Step 2: Expose Application

```
kubectl expose deployment my-app --type=LoadBalancer
--port=80
```

Step 3: Check Service

```
kubectl get services
```

External IP will be generated.

☸ Managing Clusters in Kubernetes (GKE)

1□.Introduction to Kubectl

kubectl is a command-line tool used to manage Kubernetes clusters.

It allows you to:

- Deploy applications
- View cluster resources
- Scale applications
- Check logs
- Delete resources

☞ In simple words:
kubectl = Control tool for Kubernetes.

◆ Basic kubectl Commands

Check cluster nodes:

```
kubectl get nodes
```

Check running pods:

```
kubectl get pods
```

View services:

```
kubectl get services
```

Describe resource:

```
kubectl describe pod pod-name
```

Delete deployment:

```
kubectl delete deployment my-app
```

☞ kubectl communicates with the Kubernetes API server.

2.GKE Utilization, Observability, and Cost Optimization

◆ Utilization (Resource Usage)

In GKE, you should monitor:

- CPU usage
- Memory usage
- Disk usage
- Pod usage

You can check via:

- Google Cloud Console
- Cloud Monitoring
- kubectl commands

◆ Observability (Monitoring & Logging)

Observability means understanding:

- How your application is performing
- If there are errors
- If scaling is required

GKE integrates with:

- Cloud Monitoring
- Cloud Logging

You can:

- View metrics
- Set alerts
- Track application health

◆ **Cost Optimization in GKE**

Ways to reduce cost:

✅ Use Autopilot mode for automatic optimization
✅ Enable cluster autoscaling
✅ Use smaller machine types
✅ Use Spot VMs for non-critical workloads
✅ Delete unused clusters
✅ Set resource requests & limits properly

👉 Over-provisioning increases cost.

3.Viewing Logs in a Kubernetes Cluster

Logs help you troubleshoot issues.

◆ **View Logs using kubectl**

Check pod logs:

```
kubectl logs pod-name
```

If multiple containers:

```
kubectl logs pod-name -c container-name
```

◆ **View Logs in Cloud Console**

1. Go to **Kubernetes Engine**
2. Select Cluster
3. Click **Workloads**
4. Select Pod
5. Click **Logs**

◆ **Why Logs are Important?**

- Debug application errors
- Monitor crashes
- Check performance
- Track user activity

🚀 Introduction to Google App Engine

1□.Overview of App Engine

Google App Engine (GAE) is a **Platform as a Service (PaaS)** that allows you to deploy applications without managing servers.

☞ You only focus on writing code.
Google manages:

- Infrastructure
- Scaling
- Load balancing
- Updates

Supported languages:

- Python
- Java
- Node.js

- Go
- PHP
- Ruby

In simple words:
App Engine = Deploy apps without managing servers.

2.App Engine Standard vs Flexible

App Engine has two environments:

◆ Standard Environment

- Fully managed
- Very fast startup
- Automatic scaling
- Free tier available
- Limited runtime customization

Best for:

- Lightweight web apps
- APIs
- Small to medium apps

◆ Flexible Environment

- Runs in Docker containers
- More customization
- Supports custom runtimes
- Slower startup compared to Standard
- Good for complex applications

Best for:

- Apps needing special libraries
- Long-running processes
- More control over environment

3 Deploying an Application using Cloud Shell

Step 1: Initialize App Engine

```
gcloud app create
```

Choose region.

Step 2: Deploy Application

Inside your app folder:

```
gcloud app deploy
```

Step 3: Open Application

```
gcloud app browse
```

Your app will be live.

☞ App Engine automatically handles infrastructure.

---

4 Scaling App Engine Applications

App Engine automatically scales based on traffic.

◆ Types of Scaling (Standard):

1. **Automatic Scaling**
   - Adds/removes instances based on load
   - Default option
2. **Basic Scaling**
   - Instances shut down when idle
3. **Manual Scaling**
   - Fixed number of instances

Flexible Scaling

- Automatically adds VM instances
- Supports load balancing

5□.Splitting Traffic between Application Versions

App Engine allows multiple versions of your app.

You can:

- Deploy new version
- Test it
- Split traffic between old and new versions

Example:

- Version 1 → 80% traffic
- Version 2 → 20% traffic

Useful for:

- Testing new features
- Gradual rollout
- A/B testing

Traffic splitting can be:

- Random
- Based on IP
- Based on cookie

⚡ Introduction to Cloud Functions (Google Cloud)

1□.Overview of Cloud Functions

Cloud Functions is a **serverless computing service** in Google Cloud.

It allows you to:

- Run small pieces of code
- Without managing servers
- Automatically scale
- Pay only when code runs

☞ You write the function, Google runs it.

◆ **Key Features:**

- Event-driven
- Auto scaling
- Supports multiple languages (Node.js, Python, Go, Java)
- Pay per execution

◆ **What is Event-Driven?**

Cloud Functions run when an event happens.

Examples:

- File uploaded to Cloud Storage
- Message sent to Pub/Sub
- HTTP request received

☞ Cloud Function runs only when triggered.

2.Creating Cloud Functions using Cloud Console

Steps:

1. Go to **Cloud Functions**
2. Click **Create Function**
3. Choose:
    - Function name
    - Region
    - Trigger type (HTTP or Event)
4. Select Runtime (e.g., Python)
5. Write your code
6. Click **Deploy**

After deployment:

- If HTTP trigger → You get a URL

- You can test it from browser

☞ Easy method for beginners (GUI-based).

3.Cloud Functions Receiving Events from Pub/Sub

Cloud Pub/Sub is a messaging service.

Flow:

```
Pub/Sub Topic → Message → Cloud Function → Process
```

Steps using Cloud Console:

1□.Create Pub/Sub Topic
2□.Create Cloud Function
3□.Choose Trigger = Pub/Sub
4□.Select Topic
5□.Deploy

Now:

- When a message is published to the topic
- Cloud Function automatically runs

☞ Used in event-driven architecture.

4.Cloud Functions Triggered by Storage Events (Using Cloud Shell)

Cloud Function can run when:

- File is uploaded
- File is deleted
- File is updated

Example Using Cloud Shell

Create function:

```
gcloud functions deploy my-function \
--runtime=python39 \
--trigger-bucket=my-bucket \
--entry-point=hello_gcs
```

What happens:

- When file is uploaded to **my-bucket**
- Function runs automatically

Common Storage Triggers:

- Object finalize (file upload)
- Object delete
- Object metadata update

Cloud Run – Google Cloud

1□.Introduction to Cloud Run

Cloud Run is a **fully managed serverless platform** that runs **containerized applications**.

☞ You deploy a Docker container, and Google handles:

- Infrastructure
- Scaling
- Load balancing
- Security

In simple words:
Cloud Run = Run containers without managing servers.

◆ **Key Features:**

- Serverless (no server management)
- Auto scaling (even scale to zero)
- Pay only when running
- Supports any language (if containerized)
- HTTP-based service

◆ **When to Use Cloud Run?**

- Web applications
- APIs
- Microservices
- Event-driven apps

2.Creating a Service in Cloud Run

A **Service** in Cloud Run runs continuously and responds to HTTP requests.

Steps using Cloud Console:

1□.Go to **Cloud Run**
2□.Click **Create Service**
3□.Choose:

- Service name
- Region
- Container image (from Artifact Registry or Docker Hub)

4□. Configure:

- Allow unauthenticated access (if public)

5□.Click **Deploy**

After deployment:

- You get a public URL
- Application is live

Example using Cloud Shell:

```
gcloud run deploy my-service \
--image=gcr.io/cloudrun/hello \
--platform=managed \
--region=us-central1 \
--allow-unauthenticated
```

3□.Creating a Job in Cloud Run

A **Cloud Run Job** is used for:

- Background tasks
- Batch processing
- Scheduled tasks

Unlike services:

- Jobs do not run continuously
- They run and then stop

Example Use Cases:

- Data processing
- Report generation
- File conversion
- Database cleanup

Steps using Cloud Console:

1□.Go to **Cloud Run**
2□.Click **Create Job**
3□.Select container image
4□.Configure execution settings
5□.Deploy

To run job manually:

- Click **Execute**

Example using Cloud Shell:

Create job:

```
gcloud run jobs create my-job \
--image=gcr.io/project-id/job-image \
--region=us-central1
```

Run job:

```
gcloud run jobs execute my-job --region=us-central1
```

☁ Review of Cloud Compute Options (GCP)

Google Cloud provides multiple compute services.
Choosing the correct one depends on **control, scalability, complexity, and management level**.

◆ 1□.Compute Engine

What it is:
Virtual Machines (VMs) running in the cloud.

You manage:

- OS
- Software
- Scaling
- Security updates

Best for:

- Full control over server
- Custom software installation

- Legacy applications
- Lift-and-shift migrations

☞ Think: “I want full control over my server.”

◆ 2□.Kubernetes Engine (GKE)

What it is:
Managed Kubernetes service for containers.

You manage:

- Containers
- Kubernetes configuration

Google manages:

- Control plane
- Infrastructure

Best for:

- Microservices
- Container-based applications
- Large scalable systems

☞ Think: “I want container orchestration.”

◆ 3□.App Engine

What it is:
Platform-as-a-Service (PaaS).

You manage:

- Application code only

Google manages:

- Servers

- Scaling
- Infrastructure

Best for:

- Web apps
- APIs
- Developers who don’t want server management

☞ Think: “I only want to deploy code.”

◆ 4□.Cloud Functions

What it is:
Serverless event-driven function.

Runs when:

- HTTP request
- Pub/Sub message
- Storage event

Best for:

- Small tasks
- Automation
- Background processing
- Event-driven apps

☞ Think: “I want to run small code when something happens.”

How to Choose Between Compute Options

Ask these questions:

✅ 1. Do you need full control of OS?

➞ Choose **Compute Engine**

✅ 2. Are you using containers?

➞ Choose **GKE**

✅ 3. Do you just want to deploy code easily?

➞ Choose **App Engine**

✅ 4. Is it small event-driven logic?

➞ Choose **Cloud Functions**

🎁 Planning & Configuration – Storage in Google Cloud

Storage selection depends on:

- Type of data
- Structure of data
- Scale (regional or global)
- Performance requirements

1□.Types of Storage Systems

There are mainly 3 types of storage:

◆ Object Storage

- Stores files as objects
- Example: Images, videos, backups
- Service: Cloud Storage

◆ **Block Storage**

- Used for Virtual Machines
- Acts like a hard disk
- Example: Persistent Disk

◆ **File Storage**

- Shared file system
- Used by multiple systems
- Example: Filestore

◆ **Database Storage**

- Structured or unstructured data
- SQL or NoSQL

📂 Cloud Storage

2□.Introduction to Cloud Storage

Cloud Storage is **object storage service** in Google Cloud.

Used for:

- Storing images
- Backups
- Logs
- Static website files

Storage Classes:

- Standard → Frequent access
- Nearline → Monthly access
- Coldline → Rare access
- Archive → Long-term backup

☞ Highly durable and scalable.

3.Versioning and Object Lifecycle Management

◆ **Versioning**

Keeps older versions of files when updated or deleted.

Useful for:

- Backup
- Accidental deletion recovery

◆ **Lifecycle Management**

Automatically moves or deletes objects based on rules.

Example:

- Move file to Coldline after 30 days
- Delete after 1 year

☞ Helps reduce cost.

🗄 Relational Databases (SQL)

4□.Cloud SQL (Regional Relational Storage)

Cloud SQL is a **managed relational database**.

Supports:

- MySQL
- PostgreSQL
- SQL Server

Best for:

- Traditional applications
- Regional applications

Features:

- Automated backups
- High availability (regional)
- Easy scaling

5.Cloud Spanner (Global Relational Storage)

Cloud Spanner is a **globally distributed relational database**.

Best for:

- Large-scale applications
- Global users
- Financial systems

Features:

- Strong consistency
- Horizontal scaling
- Global replication

☞ Combines SQL + global scale.

6.AlloyDB

AlloyDB is a **high-performance PostgreSQL-compatible database**.

Best for:

- Enterprise workloads
- High performance
- PostgreSQL migration

Features:

- Faster than standard PostgreSQL
- Fully managed
- High availability

7□.Cloud Datastore

Cloud Datastore is a **NoSQL document database**.

Used for:

- Scalable web applications
- Semi-structured data

Automatically scales.

8.Cloud Datastore (Second Gen)

Cloud Datastore is now part of **Firestore in Datastore mode**.

It provides:

- Better scalability
- Better performance
- Improved reliability

9.Cloud Firestore (Document Database)

Cloud Firestore is a **NoSQL document database**.

Data structure:

```
Collection → Document → Fields
```

Best for:

- Mobile apps
- Real-time apps
- Web applications

Features:

- Real-time synchronization
- Automatic scaling
- Strong consistency

10.Flexible Schemas in Cloud Firestore

Firestore allows **flexible schema**.

That means:

- Documents can have different fields
- No fixed table structure

Example:
User 1:

```
name: John
age: 25
```

User 2:

```
name: Sarah
email: sarah@email.com
```

☞ No need to predefine columns.

11.Cloud Bigtable

Cloud Bigtable is a **NoSQL wide-column database**.

Best for:

- Big data
- IoT data
- Analytics
- Time-series data

Used by:

- Large-scale applications
- High throughput systems

☞ Handles billions of rows efficiently.

🎁 Review of GCP Storage Options

Google Cloud provides multiple storage solutions.
Choosing the right one depends on:

- Type of data (structured / unstructured)
- Access frequency
- Performance needs
- Scale (regional or global)
- Cost considerations

1.Choosing a Storage Solution

◆ Step 1: Is it File or Database?

✅ If storing files (images, videos, backups)

➞ Use **Cloud Storage**

- Object storage
- Highly durable
- Lifecycle rules for cost saving

◆ Step 2: Is it Structured Data (Tables, SQL)?

✅ Regional relational database

➞ Use **Cloud SQL**

- MySQL, PostgreSQL, SQL Server

✅ **Global relational database**

➡ Use **Cloud Spanner**

- Strong consistency
- Global replication
- Massive scalability

✅ **High-performance PostgreSQL**

➡ Use **AlloyDB**

- Enterprise workloads
- PostgreSQL-compatible

◆ Step 3: Is it NoSQL Data?

✅ **Mobile / Web apps (JSON-like documents)**

➡ Use **Cloud Firestore**

- Real-time updates
- Flexible schema
- Auto scaling

✅ **Large-scale analytics / IoT / time-series**

➡ Use **Cloud Bigtable**

- Massive throughput
- Billions of rows
- Big data workloads

◆ Step 4: Is it VM Storage?

➡ Use **Persistent Disk** (block storage)

📊 Overview of Big Data Services – Google Cloud

Big Data services help you:

- Collect data
- Process data
- Store data
- Analyze data
- Automate workflows

◆ 1.Introduction to Cloud Pub/Sub – Part 1

Cloud Pub/Sub is a messaging service.

It allows applications to:

- Send messages
- Receive messages asynchronously

How it Works:

```
Publisher → Topic → Subscriber
```

- Publisher sends message
- Topic stores message
- Subscriber receives message

Used for:

- Event-driven systems
- Real-time streaming
- Microservices communication

2.Cloud Pub/Sub – Part 2 (Advanced Concepts)

Key Components:

- Topic → Message channel
- Subscription → Receives messages
- Pull subscription → Subscriber pulls message
- Push subscription → Pub/Sub pushes message

Features:

- Highly scalable
- At-least-once delivery
- Global service

Used with:

- Cloud Functions
- Dataflow
- BigQuery

3.Introduction to Cloud Dataproc

Cloud Dataproc is a managed Hadoop & Spark service.

Used for:

- Batch processing
- Big data analytics
- Running Spark jobs

Advantages:

- Quick cluster creation
- Lower cost
- Integrated with BigQuery

☞ Best for Hadoop/Spark workloads.

4.Introduction to Cloud Dataflow

Cloud Dataflow is a fully managed stream & batch data processing service.

Based on:

- Apache Beam

Used for:

- Real-time analytics
- ETL pipelines
- Streaming data

Advantages:

- Auto scaling
- Serverless
- No cluster management

☞ Best for streaming pipelines.

5.Introduction to Cloud Transfer

Used for transferring large data.

Types:

- Storage Transfer Service → Transfer between cloud storage
- Transfer Appliance → Physical device for large data
- BigQuery Data Transfer Service → Scheduled data import

☞ Used for data migration.

6.Introduction to BigQuery

BigQuery is a fully managed data warehouse.

Used for:

- SQL analytics
- Large datasets
- Business intelligence

Features:

- Serverless
- Petabyte scale
- Very fast SQL queries

☞ Used for reporting & analytics.

7.Introduction to BigQuery ML

BigQuery ML allows you to:

- Build ML models using SQL

Example:

```
CREATE MODEL my_model AS SELECT ...
```

Used for:

- Prediction
- Forecasting
- Classification

☞ No separate ML tool needed.

8.BigQuery and IAM

IAM controls:

- Who can view datasets
- Who can run queries
- Who can create tables

Common Roles:

- BigQuery Admin
- BigQuery Data Viewer
- BigQuery Data Editor

☞ Secure access to analytics data.

9.Introduction to Cloud Composer

Cloud Composer is a managed Apache Airflow service.

Used for:

- Workflow orchestration
- Scheduling data pipelines

Example:

- Daily ETL job
- Scheduled reporting

10.Cloud Composer Architecture

Components:

- Airflow Web Server
- Scheduler
- Workers
- Cloud SQL (metadata database)
- GKE cluster

Workflow:

```
DAG → Scheduler → Workers → Execute tasks
```

11.Cloud Memorystore

Cloud Memorystore is a managed in-memory database.

Supports:

- Redis
- Memcached

Used for:

- Caching
- Session storage

12.Introduction to Cloud Data Fusion

Cloud Data Fusion is a fully managed ETL service.

Used for:

- Data integration
- Data transformation
- Creating pipelines visually

Features:

- Drag-and-drop interface
- Connects to multiple data sources
- Built on CDAP

☞ Good for data engineers.

🌐 Networking in the Cloud – VPCs & VPNs (Google Cloud)

Cloud networking allows secure communication between:

- Cloud resources
- On-premises data centers
- Users and applications

◆ 1.Introduction to Networking in the Cloud

In Google Cloud, networking is:

- Software-defined
- Global
- Highly scalable

Main components:

- VPC (Virtual Private Cloud)
- Subnets

- Firewall rules
- Routes
- VPN
- Load balancers

◆ 2.Overview of Virtual Private Clouds (VPC)

A **VPC** is a private network inside Google Cloud.

It allows you to:

- Launch VMs
- Control IP ranges
- Configure firewall rules
- Connect securely to other networks

Key Features:

- Global resource (spans regions)
- Subnets are regional
- Custom or auto mode

VPC Modes:

- **Auto Mode** → Subnets created automatically
- **Custom Mode** → You create subnets manually

☞ Best practice: Use **Custom Mode** in production.

3.Creating a Virtual Private Cloud

Steps:

1. Go to VPC Network
2. Click Create VPC
3. Choose:
    - Name
    - Mode (Auto/Custom)
    - Subnet ranges
    - Firewall rules

Important:

- Define IP ranges carefully
- Avoid overlapping IP ranges (especially if using VPN)

4.Overview of Virtual Private Networks (VPN)

A **Cloud VPN** securely connects:

- Cloud VPC ↔ On-premises network

It uses:

- IPsec encryption

Types:

- Classic VPN
- HA VPN (High Availability) ✅ (Recommended)

HA VPN:

- 99.99% SLA
- Redundant tunnels

5.Implementing a Virtual Private Network

To set up Cloud VPN:

Requirements:

- VPC network
- External IP address
- VPN gateway
- On-premises VPN device

Steps:

1. Create VPN Gateway
2. Configure tunnels
3. Set up routing
4. Configure firewall rules

Routing types:

- Static routing
- Dynamic routing (BGP) → Recommended

6.Cloud and On-Premises Network Integration

Ways to connect Cloud & On-Prem:

1□.Cloud VPN

- Encrypted connection over internet
- Good for medium workloads

2□.Dedicated Interconnect

- Direct physical connection
- High bandwidth
- Low latency

3□.Partner Interconnect

- Through service provider

🌐 Networking in the Cloud

IP Addressing, Firewalls, Subnets, Load Balancing & DNS (Google Cloud)

🔹 1.IP Addressing and CIDR Blocks

Every VM in a VPC gets:

- Internal IP (private)
- Optional External IP (public)

📌 CIDR (Classless Inter-Domain Routing)

CIDR defines IP ranges.

Example:

```
10.0.0.0/24
```

- `/24` → 256 IP addresses
- Format: `IP / Prefix`

2. Implementing Firewall Rules

Firewall rules control traffic in a VPC.

Google Cloud firewalls are:

- Stateful
- Network-level

📌 Firewall Rule Components

- Direction → Ingress / Egress
- Action → Allow / Deny
- Targets → All instances / Tags / Service accounts
- Protocols & Ports → TCP, UDP, ICMP

Example:

Allow HTTP:

- Direction: Ingress
- Port: TCP 80
- Target tag: web-server

📌 **Priority**

- Lower number = Higher priority
- Deny overrides allow if priority is higher

3.Overview of Load Balancing

Load balancing distributes traffic across multiple servers.

Benefits:

- High availability
- Fault tolerance
- Scalability

Google Cloud Load Balancing is:

- Global
- Fully managed

📌 Types of Load Balancers

🌍 External Load Balancer

- For internet traffic
- Global HTTP(S) LB

🏢 Internal Load Balancer

- Inside VPC only

Network Load Balancer

- TCP/UDP traffic

4.Implementing Load Balancing

Steps:

1□.Create instance group
2□.Create backend service
3□.Configure health check

4□.Create frontend (IP + Port)
5□.Create forwarding rule

5.Implementing DNS in Google Cloud

Google Cloud provides **Cloud DNS**.

DNS translates:

```
Domain name → IP address
```

Example:

```
www.example.com → 34.120.10.5
```

📌 Types of DNS Zones

🌍 Public Zone

- Accessible from internet

🔒 Private Zone

- Only accessible inside VPC

🚀 Deployment Manager – Google Cloud

◆ What is Deployment Manager?

Deployment Manager is an Infrastructure as Code (IaC) tool in Google Cloud.

It allows you to:

- Create
- Configure
- Manage
- Update

Google Cloud resources using configuration files.

☞ Instead of creating resources manually, you define them in a template.

Why Use Deployment Manager?

Benefits:

- Automated deployments
- Repeatable environments
- Version-controlled infrastructure
- Easy updates & rollbacks

Good for:

- Dev / Test / Prod environments
- Large infrastructure setups

How Deployment Manager Works

You write a configuration file (YAML or Python template).

Example resources:

- VM instances
- VPC networks
- Firewalls
- Load balancers
- Cloud SQL

Then deploy using:

```
gcloud deployment-manager deployments create
```

Key Components

1□.Configuration File (YAML)

Defines resources to create.

2□.Templates

Reusable resource definitions.

3□.Deployment

A collection of resources created together.

Basic Deployment Flow

```
Write YAML → Run gcloud command → Resources created
```

◆ Example (Conceptual YAML)

```
resources:
- name: my-vm
  type: compute.v1.instance
```

This creates a VM instance.

Updating a Deployment

To modify infrastructure:

```
gcloud deployment-manager deployments update
```

Deployment Manager:

- Adds new resources
- Updates existing ones
- Deletes removed resources

Deleting a Deployment

```
gcloud deployment-manager deployments delete
```

This removes all resources created in that deployment.

.

🚀 Deploying Applications Using Deployment Manager

You can deploy:

- Web applications
- Multi-tier architecture
- Microservices setup

📌 Example Scenario

Deploy a web application:

1□.Create VPC
2□.Create firewall rule (allow HTTP)
3□.Create VM or instance group
4□. Attach load balancer

All defined in one YAML file.

📌 Typical Architecture

```
VPC
 ├── Firewall
 ├── Instance Group
 └── Load Balancer
```

All deployed automatically with one command.

🔥 Advantages

✓ Infrastructure as Code
✓ Automated provisioning
✓ Easy environment replication
✓ Integrated with gcloud

📊 Observability in Google Cloud

Monitoring, Logging & Tracing with Cloud Operations Suite

Observability helps you:

- Monitor performance
- Detect issues
- Troubleshoot problems
- Improve reliability

1.Cloud Operations Suite

Cloud Operations Suite (formerly Stackdriver) includes:

- Cloud Monitoring
- Cloud Logging
- Cloud Trace
- Error Reporting
- Profiler

It provides:

- Metrics
- Logs
- Traces
- Dashboards
- Alerts

☞ Used to monitor applications and infrastructure.

◆ 2.Metrics Explorer

Metrics Explorer is part of Cloud Monitoring.

Used to:

- View system metrics
- Create charts
- Analyze performance

Example Metrics:

- CPU usage
- Memory usage
- Disk I/O
- Network traffic

You can:

- Filter by resource
- Create dashboards
- Set alert policies

◆ 3.Installing Cloud Monitoring Agent

By default:

- Basic metrics are collected automatically.

To collect:

- Memory usage
- Disk metrics
- Application metrics

You must install the **Cloud Monitoring Agent** on VMs.

Installation:

Run script via SSH on VM:

```
Install monitoring agent
```

☞ Required for advanced metrics.

◆ 4.Cloud Logging

Cloud Logging stores and analyzes logs.

Sources:

- VM instances
- GKE
- App Engine
- Cloud Functions
- Load balancers

Types of logs:

- Admin Activity logs
- Data Access logs
- System logs
- Application logs

◆ 5.Logs Explorer

Logs Explorer allows you to:

- Search logs
- Filter logs
- Create log-based metrics
- Troubleshoot errors

Example filter:

```
resource.type="gce_instance"
severity="ERROR"
```

You can:

- Create alerts from logs
- Export logs to BigQuery or Cloud Storage

◆ 6.Cloud Trace

Cloud Trace is used for:

- Distributed tracing
- Latency analysis

It shows:

- How long each service takes
- Bottlenecks in microservices

Useful for:

- GKE applications
- Microservices architecture

◆ 7.The Four Golden Signals (Very Important for Exams ★)

Defined by Google SRE.

These are 4 key metrics to monitor system health:

Signal	Meaning
Latency	Time to process request
Traffic	Number of requests
Errors	Failed requests
Saturation	Resource usage (CPU, memory)

📌 Example

If website is slow:

- Check Latency
- Check CPU (Saturation)

- Check Error rate

Observability Architecture Example

```
Application
   ↓
Monitoring Agent
   ↓
Cloud Monitoring → Dashboard & Alerts
Cloud Logging → Logs Explorer
Cloud Trace → Latency Analysis
```

🔥 Exam Tips

✓ Monitoring = Metrics
✓ Logging = Log data
✓ Trace = Latency & request flow
✓ Install Monitoring Agent for detailed VM metrics
✓ Remember Four Golden Signals